

Stáhnutí dat, příprava nejbarevnějších obrázků, příprava a trénink modelu

```

import os
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras.layers import Input, Conv2D, MaxPooling2D, UpSampling2D, concatenate, LeakyReLU, Flatten,
from tensorflow.keras.models import Model
from tensorflow.keras.preprocessing.image import img_to_array, load_img
from skimage.color import rgb2lab, lab2rgb
from scipy.ndimage import gaussian_filter

# --- 1. STAŽENÍ A PŘÍPRAVA DAT ---
FILE_ID = "1unRRivEqnmhJloTxA-Ls8CJrtAbk24PY"
print("Stahuji dataset aut...")
!gdown --id {FILE_ID} -O cars_dataset.zip
print("Rozbaluji data...")
!unzip -q -o cars_dataset.zip -d car_dataset

IMG_SIZE = 128
dataset_path = 'car_dataset'

print("Načítám obrázky a vybírám ty nejbarevnější...")
vsechny_obrazky = []
barevnosti = []

count = 0
for root, dirs, files in os.walk(dataset_path):
    for file in files:
        if file.lower().endswith(('.png', '.jpg', '.jpeg')):
            try:
                img_path = os.path.join(root, file)
                img = load_img(img_path, target_size=(IMG_SIZE, IMG_SIZE))
                img_array = img_to_array(img) / 255.0
                lab_img = rgb2lab(img_array)
                skore = np.sum(np.abs(lab_img[:, :, 1:]))
                vsechny_obrazky.append(lab_img)
                barevnosti.append(skore)
                count += 1
                if count >= 1000: break
            except: continue
    if count >= 1000: break

Pocet_nejlepsich = 300
nejlepsi_indexy = np.argsort(barevnosti)[-Pocet_nejlepsich:]

X = []
Y = []
for idx in nejlepsi_indexy:
    lab = vsechny_obrazky[idx]
    X.append(lab[:, :, 0:1] / 100.0)
    Y.append(lab[:, :, 1:] / 128.0)

X = np.array(X, dtype=np.float32)
Y = np.array(Y, dtype=np.float32)

# Z dat uděláme TensorFlow Dataset pro plynulejší custom trénink
batch_size = 16
train_dataset = tf.data.Dataset.from_tensor_slices((X, Y)).shuffle(300).batch(batch_size)

# --- 2. ARCHITEKTURA SÍTÍ ---

# A) GENERÁTOR (Náš starý známý U-Net - "Malíř")
def build_generator():
    inputs = Input(shape=(IMG_SIZE, IMG_SIZE, 1))
    c1 = Conv2D(32, (3, 3), activation='relu', padding='same')(inputs)
    p1 = MaxPooling2D((2, 2))(c1)
    c2 = Conv2D(64, (3, 3), activation='relu', padding='same')(p1)
    p2 = MaxPooling2D((2, 2))(c2)
    c3 = Conv2D(128, (3, 3), activation='relu', padding='same')(p2)
    u4 = UpSampling2D((2, 2))(c3)
    m4 = concatenate([u4, c2])
    c4 = Conv2D(64, (3, 3), activation='relu', padding='same')(m4)
    u5 = UpSampling2D((2, 2))(c4)
    m5 = concatenate([u5, c1])
    c5 = Conv2D(32, (3, 3), activation='relu', padding='same')(m5)
    outputs = Conv2D(2, (3, 3), activation='tanh', padding='same')(c5)
    return Model(inputs, outputs, name="Generator")

```

```

generator = build_generator()

# B) DISKRIMINÁTOR (Nová síť - "Kritik")
# Dívá se na vstupní černobílou fotku (L) a vybarvení (AB) a hádá: Pravé (1) nebo Falešné (0)?
def build_discriminator():
    # Přijímá spojený obraz: 1 kanál (L) + 2 kanály (AB) = 3 kanály
    inputs = Input(shape=(IMG_SIZE, IMG_SIZE, 3))

    x = Conv2D(64, (4, 4), strides=(2, 2), padding='same')(inputs)
    x = LeakyReLU(alpha=0.2)(x)
    x = Conv2D(128, (4, 4), strides=(2, 2), padding='same')(x)
    x = LeakyReLU(alpha=0.2)(x)
    x = Conv2D(256, (4, 4), strides=(2, 2), padding='same')(x)
    x = LeakyReLU(alpha=0.2)(x)

    x = Flatten()(x)
    x = Dense(1)(x) # Výstup je jediné číslo (skóre realističnosti)

    return Model(inputs, x, name="Discriminator")

discriminator = build_discriminator()

# --- 3. PŘÍPRAVA NA "PŘETAHOVANOU" (Ztrátové funkce a optimalizátory) ---
cross_entropy = tf.keras.losses.BinaryCrossentropy(from_logits=True)

# Jak se hodnotí Diskriminátor (musí poznat pravé od falešného)
def discriminator_loss(real_output, fake_output):
    real_loss = cross_entropy(tf.ones_like(real_output), real_output) # Chce na pravé říct 1
    fake_loss = cross_entropy(tf.zeros_like(fake_output), fake_output) # Chce na falešné říct 0
    return real_loss + fake_loss

# Jak se hodnotí Generátor (musí oklamat diskriminátor + nebýt úplně mimo realitu)
def generator_loss(fake_output, gen_output, target):
    # Generátor chce, aby si diskriminátor myslel, že falešné je pravé (1)
    gan_loss = cross_entropy(tf.ones_like(fake_output), fake_output)
    # Stále používáme MAE (L1 loss), aby obarvoval auta správně a nekreslil nesmysly
    l1_loss = tf.reduce_mean(tf.abs(target - gen_output))
    return gan_loss + (100.0 * l1_loss) # L1 je 100x důležitější než ošálení kritika

generator_optimizer = tf.keras.optimizers.Adam(2e-4, beta_1=0.5)
discriminator_optimizer = tf.keras.optimizers.Adam(2e-4, beta_1=0.5)

# --- 4. VLASTNÍ TRÉNOVACÍ SMYČKA (Nahrazuje model.fit) ---
@tf.function
def train_step(input_image, target):
    with tf.GradientTape() as gen_tape, tf.GradientTape() as disc_tape:
        # Generátor namaluje obrázek
        gen_output = generator(input_image, training=True)

        # Spojíme L vstup a AB barvy pro kritika
        real_image_concat = tf.concat([input_image, target], axis=-1)
        fake_image_concat = tf.concat([input_image, gen_output], axis=-1)

        # Kritik ohodnotí skutečné i namalované
        real_output = discriminator(real_image_concat, training=True)
        fake_output = discriminator(fake_image_concat, training=True)

        # Spočítáme pokuty
        gen_loss = generator_loss(fake_output, gen_output, target)
        disc_loss = discriminator_loss(real_output, fake_output)

    # Aplikujeme úpravy vah (učení)
    generator_gradients = gen_tape.gradient(gen_loss, generator.trainable_variables)
    discriminator_gradients = disc_tape.gradient(disc_loss, discriminator.trainable_variables)

    generator_optimizer.apply_gradients(zip(generator_gradients, generator.trainable_variables))
    discriminator_optimizer.apply_gradients(zip(discriminator_gradients, discriminator.trainable_variables))

    return gen_loss, disc_loss

EPOCHS = 25 # Pro sytější barvy doporučuji 50-100

print("Zahajuji GAN trénink. Toto chvíli potrvá...")
for epoch in range(EPOCHS):
    gen_loss_avg = 0
    disc_loss_avg = 0
    batches = 0
    for input_image, target in train_dataset:
        g_loss, d_loss = train_step(input_image, target)
        gen_loss_avg += g_loss
        disc_loss_avg += d_loss
        batches += 1

```

```
print(f"Epocha {epoch+1}/{EPOCHS} | Ztráta Generátoru: {gen_loss_avg/batches:.4f} | Ztráta Kritika: {disc_lo
```

```
Stahuji dataset aut...
/usr/local/lib/python3.12/dist-packages/gdown/__main__.py:139: FutureWarning: Option `--id` was deprecated in ver
warnings.warn(
Downloading...
From (original): https://drive.google.com/uc?id=1unRRivEqnmhjloTxA-Ls8CJrtAbk24PY
From (redirected): https://drive.google.com/uc?id=1unRRivEqnmhjloTxA-Ls8CJrtAbk24PY&confirm=t&uuiid=16e155cb-e072-
To: /content/cars_dataset.zip
100% 38.0M/38.0M [00:00<00:00, 173MB/s]
Rozbaluji data...
Načítám obrázky a vybírám ty nejbarevnější...
/usr/local/lib/python3.12/dist-packages/keras/src/layers/activations/leaky_relu.py:41: UserWarning: Argument `alp
warnings.warn(
Zahajuji GAN trénink. Toto chvíli potrvá...
Epocha 1/25 | Ztráta Generátoru: 10.1244 | Ztráta Kritika: 0.9957
Epocha 2/25 | Ztráta Generátoru: 11.2624 | Ztráta Kritika: 0.8425
Epocha 3/25 | Ztráta Generátoru: 11.8457 | Ztráta Kritika: 0.7059
Epocha 4/25 | Ztráta Generátoru: 11.1364 | Ztráta Kritika: 0.9082
Epocha 5/25 | Ztráta Generátoru: 11.2971 | Ztráta Kritika: 0.6135
Epocha 6/25 | Ztráta Generátoru: 11.7212 | Ztráta Kritika: 1.0222
Epocha 7/25 | Ztráta Generátoru: 10.6735 | Ztráta Kritika: 0.6026
Epocha 8/25 | Ztráta Generátoru: 11.3595 | Ztráta Kritika: 0.7741
Epocha 9/25 | Ztráta Generátoru: 11.2230 | Ztráta Kritika: 0.3463
Epocha 10/25 | Ztráta Generátoru: 11.3539 | Ztráta Kritika: 1.2683
Epocha 11/25 | Ztráta Generátoru: 10.6985 | Ztráta Kritika: 0.3379
Epocha 12/25 | Ztráta Generátoru: 11.3015 | Ztráta Kritika: 0.8060
Epocha 13/25 | Ztráta Generátoru: 11.6421 | Ztráta Kritika: 0.8790
Epocha 14/25 | Ztráta Generátoru: 10.5292 | Ztráta Kritika: 0.6074
Epocha 15/25 | Ztráta Generátoru: 11.1527 | Ztráta Kritika: 0.8322
Epocha 16/25 | Ztráta Generátoru: 10.7854 | Ztráta Kritika: 0.5057
Epocha 17/25 | Ztráta Generátoru: 11.5286 | Ztráta Kritika: 0.6120
Epocha 18/25 | Ztráta Generátoru: 11.1883 | Ztráta Kritika: 0.6324
Epocha 19/25 | Ztráta Generátoru: 11.5520 | Ztráta Kritika: 0.6554
Epocha 20/25 | Ztráta Generátoru: 11.0346 | Ztráta Kritika: 0.7106
Epocha 21/25 | Ztráta Generátoru: 11.4863 | Ztráta Kritika: 0.5743
Epocha 22/25 | Ztráta Generátoru: 11.4176 | Ztráta Kritika: 1.0645
Epocha 23/25 | Ztráta Generátoru: 10.8167 | Ztráta Kritika: 0.5949
Epocha 24/25 | Ztráta Generátoru: 11.3422 | Ztráta Kritika: 0.7719
Epocha 25/25 | Ztráta Generátoru: 11.1380 | Ztráta Kritika: 0.7388
```

Tvorba posuvníků pro porovnání výsledků

```
import ipywidgets as widgets
from IPython.display import display
import matplotlib.pyplot as plt
import numpy as np
from skimage.color import lab2rgb
from scipy.ndimage import gaussian_filter

# Funkce, která se zavolá při každém pohnutí posuvníkem
def interaktivni_prohlizec(index):
    # 1. Připravíme černobílý vstup
    l_vstup = X[index:index+1]

    # 2. Necháme vytrénovaný GAN model (Generátor) obarvit obrázek v reálném čase
    pred_ab = generator.predict(l_vstup, verbose=0)[0]

    # Jemné vyhlazení barev (aby nevznikaly ostré pixely)
    pred_ab[:, :, 0] = gaussian_filter(pred_ab[:, :, 0], sigma=1.0)
    pred_ab[:, :, 1] = gaussian_filter(pred_ab[:, :, 1], sigma=1.0)

    # 3. Složíme AI obarvený obrázek do RGB
    lab_ai = np.zeros((IMG_SIZE, IMG_SIZE, 3))
    lab_ai[:, :, 0] = X[index, :, :, 0] * 100
    lab_ai[:, :, 1:] = pred_ab * 128
    vysledny_lab = np.clip(lab_ai, -128, 127)
    rgb_ai = lab2rgb(vysledny_lab)

    # 4. Složíme původní originál (pro srovnání)
    lab_orig = np.zeros((IMG_SIZE, IMG_SIZE, 3))
    lab_orig[:, :, 0] = X[index, :, :, 0] * 100
    lab_orig[:, :, 1:] = Y[index] * 128
    rgb_orig = lab2rgb(lab_orig)

    # 5. Vykreslíme všechny 3 vedle sebe
    fig, ax = plt.subplots(1, 3, figsize=(16, 5))

    ax[0].imshow(X[index, :, :, 0], cmap='gray')
    ax[0].set_title(f"Vstupní data (Černobíle) | Index: {index}", fontsize=14)
    ax[0].axis('off')
```

```

ax[1].imshow(rgb_ai)
ax[1].set_title("Obarveno AI (GAN Generátor)", fontsize=14, color='green')
ax[1].axis('off')

ax[2].imshow(rgb_orig)
ax[2].set_title("Skutečný originál", fontsize=14, color='blue')
ax[2].axis('off')

plt.tight_layout()
plt.show()

# Vytvoření grafického posuvníku (od 0 do počtu obrázků v datasetu)
posuvnik = widgets.IntSlider(
    value=0,
    min=0,
    max=len(X)-1,
    step=1,
    description='Obrázek č.:',
    layout=widgets.Layout(width='800px')
)

# Propojení posuvníku s naší funkcí
print("Interaktivní prohlížeč spuštěn. Pohni posuvníkem pro změnu obrázku:")
widgets.interact(interaktivni_prohlizec, index=posuvnik);

```

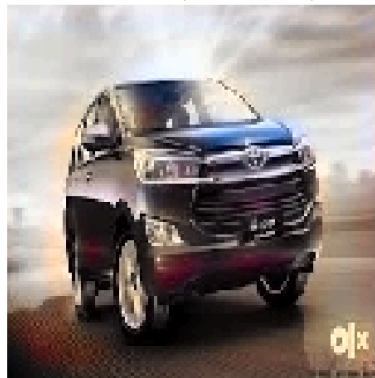
Interaktivní prohlížeč spuštěn. Pohni posuvníkem pro změnu obrázku:

Obrázek č.: 0

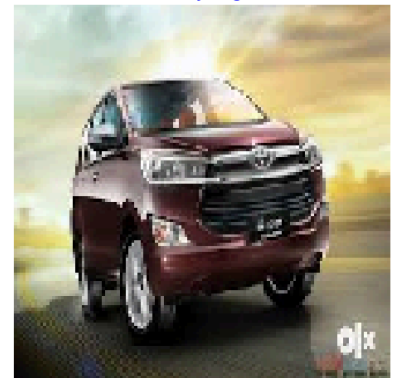
Vstupní data (Černobíle) | Index: 0



Obarveno AI (GAN Generátor)



Skutečný originál



Helper pro tvorbu prezentace

```

!pip install python-pptx -q
import numpy as np
import matplotlib.pyplot as plt
from skimage.color import lab2rgb
from scipy.ndimage import gaussian_filter
from pptx import Presentation
from pptx.util import Inches, Pt
from pptx.dml.color import RGBColor
from google.colab import files

# --- 1. FUNKCE PRO ZKOPIROVÁNÍ 3 AUT (3 řádky) NA JEDEN OBRÁZEK ---
def create_3_car_collage(indices, filename):
    fig, axes = plt.subplots(3, 3, figsize=(12, 10))
    fig.patch.set_facecolor('#0f172a') # Tmavé pozadí

    for row, idx in enumerate(indices):
        l_vstup = X[idx:idx+1]

        # Voláme GAN generátor pro obarvení
        pred_ab = generator.predict(l_vstup, verbose=0)[0]
        pred_ab[:, :, 0] = gaussian_filter(pred_ab[:, :, 0], sigma=1.0)
        pred_ab[:, :, 1] = gaussian_filter(pred_ab[:, :, 1], sigma=1.0)

        lab = np.zeros((128, 128, 3))
        lab[:, :, 0] = X[idx, :, :, 0] * 100
        lab[:, :, 1:] = pred_ab * 128
        rgb_ai = lab2rgb(np.clip(lab, -128, 127))

        orig_lab = np.zeros((128, 128, 3))

```

```

orig_lab[:, :, 0] = X[idx, :, :, 0] * 100
orig_lab[:, :, 1:] = Y[idx] * 128
rgb_orig = lab2rgb(orig_lab)

axes[row, 0].imshow(X[idx, :, :, 0], cmap='gray')
axes[row, 1].imshow(rgb_ai)
axes[row, 2].imshow(rgb_orig)

for ax in axes[row]:
    ax.axis('off')

if row == 0:
    axes[row, 0].set_title("Vstup (Jas L)", color='white', fontsize=14)
    axes[row, 1].set_title("GAN Kolorizace", color='#2dd4bf', fontsize=14)
    axes[row, 2].set_title("Skutečný cíl", color='#c084fc', fontsize=14)

plt.tight_layout()
plt.savefig(filename, facecolor=fig.get_facecolor(), dpi=150)
plt.close()

# --- 2. GENEROVÁNÍ POWERPOINTU ---
prs = Presentation()

# Funkce pro nastavení tmavého pozadí na slajd
def set_dark_bg(slide):
    background = slide.background
    fill = background.fill
    fill.solid()
    fill.fore_color.rgb = RGBColor(15, 23, 42) # Slate 900

# Snímek 1: Titulní
slide_1 = prs.slides.add_slide(prs.slide_layouts[0])
set_dark_bg(slide_1)
title = slide_1.shapes.title
title.text = "Kolorizace Aut pomocí AI"
title.text_frame.paragraphs[0].font.color.rgb = RGBColor(248, 250, 252) # Bílá
subtitle = slide_1.placeholders[1]
subtitle.text = "Od šedé k fotorealitě: Implementace architektury U-Net a GAN"
subtitle.text_frame.paragraphs[0].font.color.rgb = RGBColor(45, 212, 191) # Teal

# Snímek 2: Model Architecture
slide_2 = prs.slides.add_slide(prs.slide_layouts[1])
set_dark_bg(slide_2)
slide_2.shapes.title.text = "Jak model přemýšlí: Architektura Sítě"
slide_2.shapes.title.text_frame.paragraphs[0].font.color.rgb = RGBColor(45, 212, 191)
tf2 = slide_2.shapes.placeholders[1].text_frame
tf2.text = "Krok 1: ENKODÉR"
tf2.paragraphs[0].font.color.rgb = RGBColor(192, 132, 252) # Purple
p2_1 = tf2.add_paragraph()
p2_1.text = "Model skenuje obrázek, zmenšuje ho a hledá hrany kapoty a kol."
p2_1.font.color.rgb = RGBColor(203, 213, 225)
tf2.add_paragraph().text = "Krok 2: DEKODÉR A ZKRATKY (Skip Connections)"
tf2.paragraphs[2].font.color.rgb = RGBColor(192, 132, 252)
p2_2 = tf2.add_paragraph()
p2_2.text = "Aby barva nepřetekla mimo okraje, síť si zkratkou pošle ostré hrany z enkodéru přímo k vybarvování."
p2_2.font.color.rgb = RGBColor(203, 213, 225)
tf2.add_paragraph().text = "Krok 3: DISKRIMINÁTOR (Kritik)"
tf2.paragraphs[4].font.color.rgb = RGBColor(192, 132, 252)
p2_3 = tf2.add_paragraph()
p2_3.text = "Nový AI kritik hodnotí, jak moc auto vypadá reálně, a nutí generátor dělat sytější barvy."
p2_3.font.color.rgb = RGBColor(203, 213, 225)

# Snímek 3: Bias Loss
slide_3 = prs.slides.add_slide(prs.slide_layouts[1])
set_dark_bg(slide_3)
slide_3.shapes.title.text = "Grayscale Bias & Optimalizace"
slide_3.shapes.title.text_frame.paragraphs[0].font.color.rgb = RGBColor(45, 212, 191)
tf3 = slide_3.shapes.placeholders[1].text_frame
tf3.text = "Proč čistý U-Net barví do šeda?"
tf3.paragraphs[0].font.color.rgb = RGBColor(251, 113, 133) # Coral
p3_1 = tf3.add_paragraph()
p3_1.text = "Základní MAE (Mean Absolute Error) penalizuje chyby. U-Net se bojí tipnout špatnou barvu a volí raději šedé."
p3_1.font.color.rgb = RGBColor(203, 213, 225)
tf3.add_paragraph().text = "Řešení s využitím GAN"
tf3.paragraphs[2].font.color.rgb = RGBColor(251, 113, 133)
p3_2 = tf3.add_paragraph()
p3_2.text = "Zavedením Binary Crossentropy (Diskriminátoru) síť nezajímá jen matematická odchylka, ale snaha kompromisovat mezi barvou a strukturou."
p3_2.font.color.rgb = RGBColor(203, 213, 225)

print("Generuji koláž pro snímky...")
# Snímek 4: Výsledky 1 (3 auta nad sebou)
slide_4 = prs.slides.add_slide(prs.slide_layouts[5])
set_dark_bg(slide_4)

```

```

set_dark_bg(slide_4)
slide_4.shapes.title.text = "Výsledky Modelu (Část 1/2)"
slide_4.shapes.title.text_frame.paragraphs[0].font.color.rgb = RGBColor(45, 212, 191)
img1_path = "kolaze_1.png"
create_3_car_collage([19, 39, 67], img1_path) # Uprav si tyto 3 čísla (indexy z datasetu)
prs.slides[3].shapes.add_picture(img1_path, Inches(1), Inches(1.5), height=Inches(5.5))

# Snímek 5: Výsledky 2 (3 auta nad sebou)
slide_5 = prs.slides.add_slide(prs.slide_layouts[5])
set_dark_bg(slide_5)
slide_5.shapes.title.text = "Výsledky Modelu (Část 2/2)"
slide_5.shapes.title.text_frame.paragraphs[0].font.color.rgb = RGBColor(45, 212, 191)
img2_path = "kolaze_2.png"
create_3_car_collage([125, 156, 205], img2_path) # Zase, klidně změň indexy aut
prs.slides[4].shapes.add_picture(img2_path, Inches(1), Inches(1.5), height=Inches(5.5))

# Snímek 6: Závěr
slide_6 = prs.slides.add_slide(prs.slide_layouts[0])
set_dark_bg(slide_6)
title6 = slide_6.shapes.title
title6.text = "Dotazy?"
title6.text_frame.paragraphs[0].font.color.rgb = RGBColor(45, 212, 191)
subtitle6 = slide_6.placeholders[1]
subtitle6.text = "Děkuji za pozornost"
subtitle6.text_frame.paragraphs[0].font.color.rgb = RGBColor(203, 213, 225)

file_name = "Tech_Prezentace_Obarvovani_Aut.pptx"
prs.save(file_name)
print(f"\nHotovo! Stahuje se soubor {file_name}.")
files.download(file_name)

```

```

472.8/472.8 kB 28.9 MB/s eta 0:00:00
175.3/175.3 kB 16.7 MB/s eta 0:00:00

```

Generuji koláže pro snímky...

/tmp/ipykernel_762/1515374914.py:27: UserWarning: Conversion from CIE-LAB, via XYZ to sRGB color space resulted in values outside of the range [0, 1]. These values will be clipped, but may affect the appearance of the resulting image.
 rgb_ai = lab2rgb(np.clip(lab, -128, 127))

Hotovo! Stahuje se soubor Tech_Prezentace_Obarvovani_Aut.pptx.